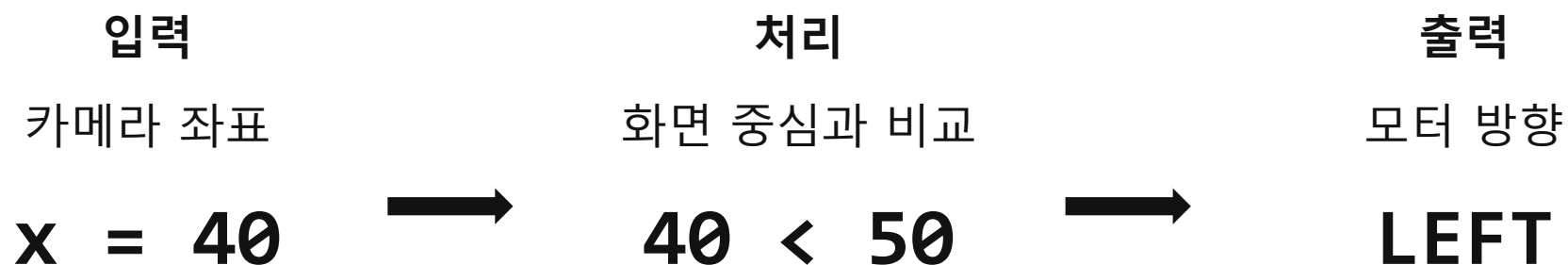




C 프로그래밍 && 깃/깃허브 기초

박준우

프로그래밍이란?



왜 C인가?



- 현대 프로그래밍 언어들의 근본
- 모든 대학 컴퓨터공학과에서 C를 기본 언어로 가르침
- 하드웨어와 가깝게 동작하는 저수준 언어로 빠른 속도
- 아두이노에서도 C기반 언어 C++를 사용

변수의 형식

자료형 이름 = 값;

`int x = 40;`

값에 이름을 붙여 다시 쓴다

기본 자료형

int

40

정수

double

0.5

소수

bool

true

참 / 거짓

char

'L'

한 문자

배열은 0부터

```
int xs[3] = {40, 50, 60};
```

`xs[0]`

40

`xs[1]`

50

`xs[2]`

60

첫 칸은 [0] · 마지막 칸은 [2]

문자와 문자열

```
char letter = 'L';
```

```
char word[] = "LEFT";
```

```
word[0] → 'L'
```

문자열은 문자를 순서대로 담은 배열

#include는 무엇일까?

#include <stdio.h>

printf · scanf

#include <stdlib.h>

표준 도구

#include <stdbool.h>

bool · true · false

printf 사용법

```
printf("Hello!\n");  
printf("%d\n", x);
```

화면에 글자와 변수 값을 출력한다

형식 문자는 값의 종류

값	printf	scanf
정수	%d	%d
소수(double)	%f	%lf
문자	%c	%c
문자열	%s	%s

₩n과 ₩0

`\n` 줄바꿈

`\0` 문자열 끝 표시

Hello, world!

실행할 것

```
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

성공하면

Hello, world!

scanf 사용법

```
int x;  
scanf("%d", &x);
```

입력한 정수를 x에 저장한다

입력값 다시 출력

실행할 것

```
#include <stdio.h>
int main(void) {
    int x;
    scanf("%d", &x);
    printf("%d\n", x);
    return 0;
}
```

성공하면

입력 40

출력 40

값을 바꿔 출력

실행할 것

```
int x = 40;
double speed = 0.5;
bool found = true;
char mode = 'L';
char word[] = "LEFT";
printf("%d %f\n", x, speed);
printf("%d %c %s\n", found, mode, word);
```

main 위에 #include <stdbool.h> 추가

성공하면

40 0.500000

1 L LEFT

사칙연산

$$5 + 2 = 7$$

$$5 - 2 = 3$$

$$5 * 2 = 10$$

$$5 / 2 = 2$$

단항연산

`i++;` → `i`를 1 늘리기

`i--;` → `i`를 1 줄이기

`-i;` → 부호 뒤집기

값 하나에 작용하는 연산

비교연산

<

작다

>=

크거나 같다

<=

작거나 같다

==

같다

>

크다

!=

다르다

논리연산

A && B

둘 다 참

A || B

하나라도 참

!A

참/거짓 뒤집기

x >= 45 && x <= 55

if / else의 형식

```
if (조건) { 실행A; }  
else if (조건) { 실행B; }  
else { 실행C; }
```

위에서부터 검사해 맞는 가지 하나를 실행

if / else로 방향 선택

실행할 것

```
int x = 40, center = 50;  
if (x < center) printf("LEFT\n");  
else if (x > center) printf("RIGHT\n");  
else printf("STOP\n");
```

성공하면

LEFT

for의 형식

```
for (시작; 조건; 증가) {  
    반복할 코드;  
}
```

시작값을 정하고 조건이 참인 동안 반복

for로 세 번 출력

실행할 것

```
for (int i = 0; i < 3; i++) {  
    printf("%d\n", i);  
}
```

성공하면

0

1

2

while의 형식

```
while (조건) {  
    반복할 코드;  
    값 바꾸기;  
}
```

조건이 참인 동안 계속 반복

while로 세 번 출력

실행할 것

```
int i = 0;
while (i < 3) {
    printf("%d\n", i);
    i++;
}
```

성공하면

0
1
2

함수는 작은 계산기

$$y = x + 1$$

입력 $x = 3$



결과 $y = 4$

C 함수의 형식

```
int plus_one(int x) {  
    return x + 1;  
}
```

int는 돌려주는 결과의 자료형

함수를 호출해 보기

실행할 것

```
int plus_one(int x) {  
    return x + 1;  
}
```

```
printf("%d\n", plus_one(3));
```

성공하면

4

방향 판단 함수

실행할 것

main 위

```
int direction(int x, int center) {  
    if (x < center) return -1;  
    if (x > center) return 1;  
    return 0;  
}
```

main 안

```
printf("%d\n", direction(40, 50));
```

성공하면

-1

통합 실습 ① 판단값

실행할 것

```
int xs[3] = {40, 50, 60};  
for (int i = 0; i < 3; i++) {  
    int d = direction(xs[i], 50);  
    printf("%d\n", d);  
}
```

성공하면

-1

0

1

통합 실습 ② 모터 방향

실행할 것

```
int xs[3] = {40, 50, 60};  
for (int i = 0; i < 3; i++) {  
    int d = direction(xs[i], 50);  
    if (d < 0) printf("LEFT\n");  
    else if (d > 0) printf("RIGHT\n");  
    else printf("STOP\n");  
}
```

성공하면

LEFT

STOP

RIGHT

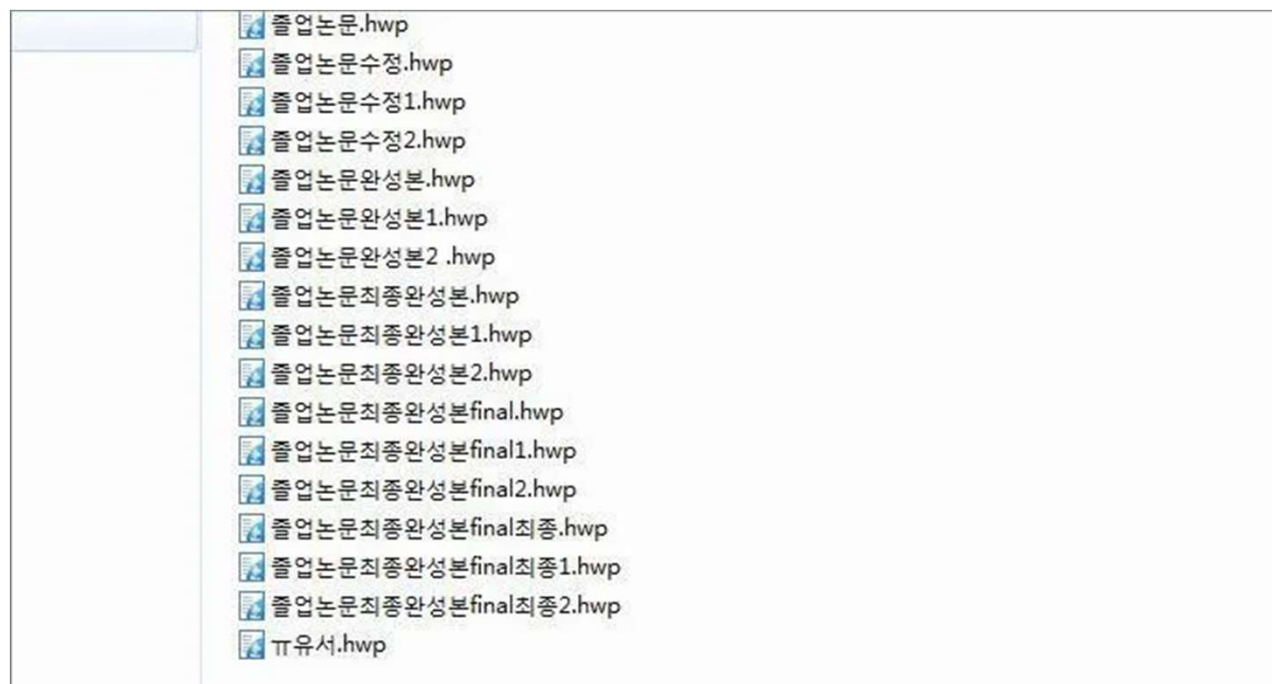
Git / Github

깃과 깃허브



Git을 쓰는 이유

Git의 핵심은 버전관리



커밋 작성자 설정

실행할 것

```
git --version
```

```
git config --global user.name "내 이름"
```

```
git config --global user.email "내 이메일"
```

성공하면

Git 버전 표시

이름.이메일 저장

사진으로 이해하는 Git



사진사 고용

촬영할 사람 모으기

사진 찍기

온라인에 올리기

사진으로 이해하는 Git

`git init`

사진사 고용

`git add .`

촬영할 사람 모으기

`git commit`

사진 찍기

`git push`

온라인에 올리기

촬영 준비와 사진 기록

git status

변경 사항 확인

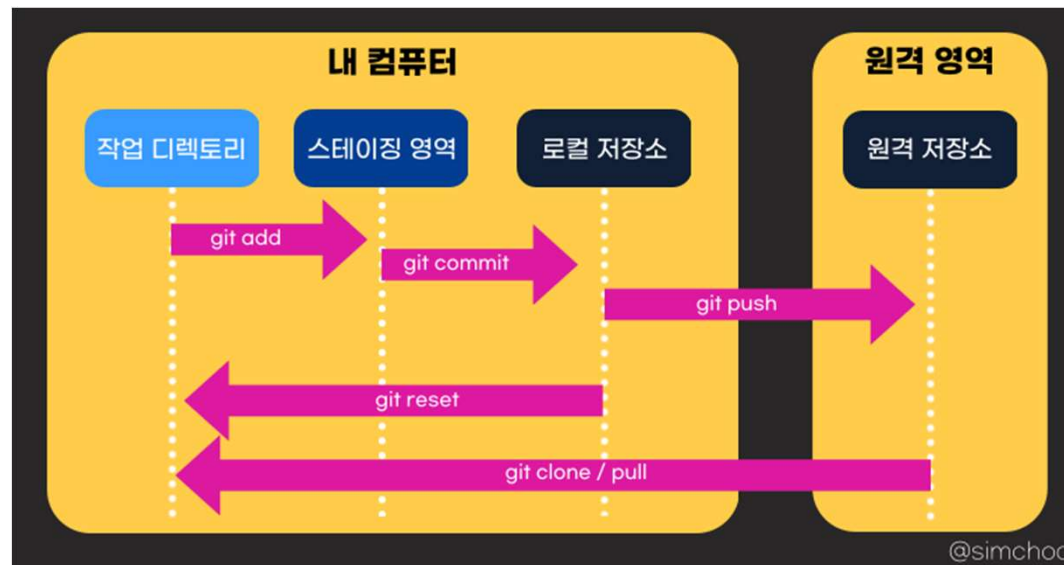
git log

찍어 둔 사진 기록

Git과 GitHub

내 컴퓨터의 Git ↔ GitHub

push → ← pull



프로필 저장소 만들기

할 것

GitHub → New repository

이름 = 본인 GitHub 아이디

Public · Add README 켜기

성공하면

프로필 README

프로필 저장소 Clone

실행할 것

```
git clone https://github.com/아이디/아이디.git
```

```
cd 아이디
```

성공하면

README.md

로컬 폴더 생성

첫 번째 사진 찍기

실행할 것

README에 한 줄 추가

```
git status
```

```
git add .
```

```
git commit -m "소개 추가"
```

성공하면

새 커밋 생성

내 PC에 기록

GitHub에 연결하기

할 것

git push 실행

브라우저에서 내 GitHub 계정 로그인

성공하면 보일 결과

GitHub README에 첫 변경 반영

두 번째 사진 찍기

실행할 것

README에 일부러 잘못된 한 줄 추가

```
git status
```

```
git add .
```

```
git commit -m "잘못된 문장 추가"
```

```
git push
```

성공하면

잘못된 문장도

기록에 남는다

사진 기록 보기

실행할 것

```
git log --oneline --graph
```

성공하면

커밋 번호

두 변경 기록

```
git log
git log --oneline //커밋 내역을 한줄씩 보여줌.
git log --graph   //커밋 내역을 시각화해서 보여줌.
```

공유한 변경 되돌리기

실행할 것

```
git revert --no-edit <커밋번호>
```

```
git push
```

```
git log --oneline --graph
```

꺾쇠 없이 본인의 커밋 번호를 입력

성공하면

잘못된 문장 삭제

원본.취소 커밋 둘 다 남
음

Reset은 강사 시연

--soft

커밋만 취소

--mixed

커밋·add 취소

--hard

파일 수정까지 폐기

시연: `git reset --soft HEAD~1`

팀 규칙

공유한 커밋은 revert
공유 전 로컬 커밋은 reset

교육용 저장소 Fork

할 것

Cire21st 원본 저장소 열기

Fork → 내 계정 선택

성공하면 보일 결과

내 계정에 저장소 사본 생성

자기 Fork를 Clone

실행할 것

```
git clone https://github.com/아이디/교육저장소.git
```

```
cd 교육저장소
```

```
git remote -v
```

성공하면

origin → 내 Fork

원본 저장소 아님

내 작업 브랜치 만들기

실행할 것

```
git switch practice/본인아이디
```

```
git status
```

성공하면

새 브랜치로 이동

practice/본인아이디

배정된 파일 수정

실행할 것

본인 파일 한 줄 수정

```
git status
```

```
git add .
```

```
git commit -m "내 파일 수정"
```

```
git push -u origin practice/본인아이디
```

성공하면

내 Fork에 브랜치 생성

수정 커밋 보임

내 Fork에 PR 만들기

할 것

내 Fork의 Pull requests → New pull request

base: 내 Fork / main

compare: practice/본인아이디

성공하면 보일 결과

내 Fork를 대상으로 PR 생성

PR 직접 Merge

할 것

변경 파일 확인

Merge pull request → Confirm merge

성공하면 보일 결과

내 Fork의 main에 변경 반영

base repository가 자기 Fork인지 다시 확인

Merge 결과 Pull

실행할 것

```
git switch main
```

```
git pull origin main
```

성공하면

로컬 main 갱신

수정 파일 확인

원본 저장소 PR 시연

할 것

cire21st 대표 사례: 내 Fork 브랜치 → 원본 main

PR 대상 저장소가 바뀌는 것 확인

성공하면 보일 결과

팀에 변경을 제안하는 흐름 이해



고생하셨습니다.

